

Lenguaje Python

Estructura de un programa ***Tipos de Datos***

Características estructurales de un programa en Python

Un programa Python consiste en un bloque con llamados (importaciones) a otros módulos y/o funciones, que ofrece el resultado de un algoritmo.

Un módulo es un conjunto de estructuras definidas en un archivo para usarlos como script o en una instancia interactiva del intérprete.

Un módulo es un archivo que contiene estructuras y declaraciones (*variables y funciones*) de Python.

Las funciones definidas de un módulo pueden ser **importadas** a otros módulos o al módulo principal.

El intérprete de Python genera archivos con extensión **.py**, y permite generar un ejecutable (**.exe**), para ser portable en cualquier equipo.

Se descargará el Intérprete correspondiente, se escribirá y se depurará el código para luego opcionalmente poder generar el ejecutable (.exe) de ser necesario, donde el compilador incluye al intérprete.

Se usará un Editor de texto o entorno IDLE, o editor SUBLIME TEXT para guardar el archivo .py.

El editor del intérprete utiliza un **prompt (>>>)** donde se escribe cada sentencia finalizada con <ENTER>, y el intérprete muestra el resultado sin prompt, o en su defecto en el área derecha del editor.

Un programa en Python puede contener varios componentes.

Podemos citar algunos de los tipos partiendo de lo más básico a lo más complejo:

1. OBJETO: Un objeto es un componente básico de una estructura que se aloja en memoria, y que tiene asociado una serie de valores y/u operaciones que se pueden realizar con él. Así pues, un objeto puede ser un string, un número int o de punto flotante (float), una lista, una tupla, un diccionario, etc... o un objeto propiamente dicho.

Se encuentran:

- a. **OBJETO:** Objetos propios de la Programación Orientada a Objetos, que no desarrollaremos en esta materia.
- b. **OBJETOS BUILT-IN:** Objetos pre-construidos por el propio lenguaje. Constituyen objetos integrados dentro del lenguaje que Python utiliza para facilitar las tareas de programación, permitiéndonos ahorrar tiempo al construir objetos o estructuras de datos automáticos; facilita la generación de estructuras complejas basadas en ellos, y consiguen un rendimiento elevado con un consumo mínimo de memoria durante el proceso de ejecución. Estos objetos son números, string, booleanos, tuplas, listas, conjuntos, diccionarios y ficheros.
- c. **OBJETO NONE:** significa "ningún objeto". Se utiliza para asignar un valor nulo a una variable determinada.

2. EXPRESIONES: Constituyen una combinación de valores, constantes, variables, operadores y funciones que son aplicadas de acuerdo a ciertas reglas de sintáxis.

3. ESTRUCTURAS: o sentencias. Son un conjunto de expresiones, que se consideran las unidades mínimas ejecutables de un programa.

4. MÓDULOS: Conjunto de sentencias ordenadas de acuerdo a un fin concreto, llamadas Funciones.

Directivas de Inicio:

```
# -*- coding: utf-8 -*-
```

Indica al sistema, la codificación de caracteres utilizado en el archivo utf-8 (podría ser cualquier codificación de caracteres).

Si no se indica una codificación de caracteres, Python podría producir un error si encontrara caracteres *extraños*:

```
print ("Se encuentro un extraño código")
```

Producirá un error de sintáxis: `SyntaxError: Non-ASCII character[...]`

En cambio, indicando el encoding correspondiente, el archivo se ejecutará correctamente.

Sangrado o Indentación

La indentación es una característica importante y obligatoria, en el uso de las estructuras de control.

Una indentación de 4 (cuatro) espacios en blanco, indicará que las instrucciones indentadas, forman parte de una misma estructura de control. Los editores la aplican en forma automática.

Una estructura de control, entonces, se definirá de la siguiente forma:

```
inicio de la estructura de control:  
    expresiones
```

Ejemplo de un programa sencillo (ejemplo.py) escrito con un editor:

```
# -*- coding: utf-8 -*- comentario de una línea  
  
""" Un programa sencillo: Dados dos números distintos,  
mostrarlos ordenados en forma creciente. """ comentario de más  
de una línea  
  
a = int (input ("Ingresar un número: "))  
b = int (input ("Ingresar otro número: "))  
if(a>b):  
    print(b,a)  
elif(b>a):  
    print(a,b)  
else:  
    print ("Los números son iguales")
```

Ejecución

Para ejecutar un módulo o programa, podemos iniciar Python y en IDLE:

File – Open – nombre módulo o programa (de extensión .py)

Run module o F5

La sentencia `import` se utiliza para importar un módulo. Se puede usar cualquier archivo de código Python como un módulo ejecutando esta sentencia en otro archivo de código Python. La sentencia `import` tiene la siguiente sintaxis:

```
>>> import os
```

```
>>> import re, datetime
```

Cuando el intérprete encuentra una sentencia `import`, este importa el módulo si el mismo está presente en la ruta de búsqueda.

Una ruta de búsqueda es una lista de directorios que el intérprete busca antes de importar un módulo.

Elementos y reglas para la construcción de un programa

a. Variables y Constantes

Python utiliza *identificadores* para denominar al programa (ejemplo), para denominar a las **funciones** dentro de un módulo (calculo), a las **variables** y a las **constantes**.

Un identificador comienza con *una letra o con guión bajo (_)* y luego sigue con *una secuencia de letras, números y guiones bajos*. Los espacios no están permitidos dentro de los identificadores.

Los siguientes son todos identificadores válidos de Python:

- suma
- suma12t
- _suma
- Suma

Python **distingue mayúsculas de minúsculas**, así que Suma es un identificador y suma es otro identificador (**por convención no comienzan con mayúsculas**). **Los nombres de los comandos se escriben en minúscula.**

Los siguientes son todos identificadores inválidos de Python:

- Suma a12t
- 8suma
- suma\%
- suma9*

Las Constantes se definen con nombres descriptivos y en mayúsculas separándolos por guiones bajos. Ejemplo:

VALOR_CONSTANTE = 12

Palabras Reservadas

Reserva 31 palabras para describir la estructura del programa, y no permite que se usen como identificadores. Esta es la lista:

and	del	from	not	while
as	elif	global	or	with
assert	else	if	pass	yield
except	import	print	break	class
exec	in	raise	continue	finally
is	return	def	for	lambda
try				

Definición de variables

Los valores de las variables en Python se asocian a objetos, según su tipo (enteros, flotantes, booleanos, string), y estos se almacenan en memoria para permitir su acceso y modificación.

a. Creación de variables:

No requiere de una declarativa específica, a través de la Estructura de Secuencia **Asignación**, queda definida con el tipo del valor asignado.

```
valor1 = 1
```

```
valor2 = 2.4
```

```
texto = "prueba"
```

```
serie = True
```

```
promedio = (valor1 + valor2) / 2
```

Podemos visualizar el tipo de variable con la función TYPE:

```
>>> type(valor2)
<class float>
```

Cuando se asigna por primera vez un valor a una variable, Python:

- crea un objeto para almacenar el valor (si no existe todavía),
- crea la variable
- asocia la variable al objeto.

El valor de la variable es el valor almacenado por el objeto.

b. Modificación de variables

Cuando se asigna un nuevo valor a una variable creada anteriormente, el valor de la variable será el nuevo valor almacenado.

Con los datos simples, las variables de Python se comportan mutables, cambian por el nuevo valor asignado, y permite que cambie el tipo de identificador, según sea el tipo del valor asignado.

Estructuras de Datos

Tipos de Datos Simples

a) Numéricos: Enteros (int) y Reales (float)

int: edad = 35

Int octal: edad = 04

int hexadecimal: edad = 0x23

float: precio = 9635.28

Operadores Aritméticos

Entre los operadores aritméticos que Python utiliza, podemos encontrar los siguientes:

Símbolo	Significado	Ejemplo	Resultado
+	Suma	a = 10 + 5	a es 15
-	Resta	a = 12 - 7	a es 5
-	Negación	a = -5	a es -5
*	Multiplicación	a = 7 * 5	a es 35
**	Exponente	a = 2 ** 3	a es 8
/	División	a = 12.5 / 2	a es 6.25

//	División entera	a = 12.5 / 2	a es 6.0
%	Módulo	a = 27 % 4	a es 3

“Siempre colocar un espacio en blanco, antes y después de un operador”

Un ejemplo sencillo con variables y operadores aritméticos:

```
monto_bruto = 175
```

```
tasa_interes = 12
```

```
monto_interes = monto_bruto * tasa_interes / 100
```

```
tasa_bonificacion = 5
```

```
importe_bonificacion = monto_bruto * tasa_bonificacion / 100
```

```
monto_netto = (monto_bruto - importe_bonificacion) + monto_interes
```

Expresiones

Una *expresión* es una porción de código Python que produce o calcula un valor (resultado):

- Mostrar el resultado de la expresión 555 es precisamente el número 555.

```
>>> 555  
555
```
- Una variable es una expresión, y el valor que produce es el que tiene asociado en el estado (si `x = 5` en el estado, entonces el resultado de la expresión `x` es el número 5).

```
>>> x=5  
>>> x  
5
```

- Evaluar una expresión no es lo mismo que mostrar un valor.

```
>>> f  
>>> frase  
'ejemplos de algoritmos'  
>>> print frase  
ejemplo de algoritmos
```

Cuando la consola de Python muestra el valor de una expresión que ha evaluado, usa el mismo formato que se usaría para introducir un valor. En el caso de las cadenas, esto implicaría que se incluyen las comillas. Pero la sentencia `print` despliega el valor de la expresión que, en este caso, es el contenido de la cadena.

b. Cadena de Texto o de Caracteres (String)

Se expresan entre comillas simples o dobles, pueden asignarse a una variable, y se muestran con la sentencia `print ()`

- Cadena de texto (*string*) 1 línea:

```
cadena_unalinea = "Comencemos... Buen suerte!"
```

- Cadena de texto (*string*) multilínea:

```
cadena_multilinea = """Esta es una cadena de varias líneas, que  
nos sirve de ejemplo: prueben hacerla con un texto interesante y  
entretenido"""
```

```
>>> print ("Bienvenidos")  
Bienvenidos  
>>> frase = "Lista de Precios"  
>>> print (frase)  
Lista de Precios  
>>> type(frase)  
<class str>
```

Operadores de String

- **Concatenar**

El proceso de *concatenación* se realiza mediante el operador de suma (+).

```
>>> mensaje1 = 'Hola' + ' ' + 'Estudiantes'  
>>> print (mensaje1)  
>>> Hola Estudiantes
```

- **Multiplicar**

Genera copias de cadenas usando el operador de multiplicación (*).

```
>>> mensaje2a = 'Hola ' * 3  
>>> mensaje2b = 'Estudiantes'  
>>> print (mensaje2a + mensaje2b)  
>>> Hola Hola Hola Estudiantes
```

- **Añadir**

Añade de manera sucesiva al final de una cadena de caracteres, a través del operador compuesto (+=).

```
>>> mensaje3 = 'Hola'
>>> mensaje3 += ' '
>>> mensaje3 += 'Estudiantes'
>>> print (mensaje3)
>>> Hola Estudiantes
```

Slices

Las cadenas de string pueden trabajarse en forma indexada, donde un índice indica un carácter, permitiendo realizar operaciones.

El índice inicial siempre se incluye, el índice final siempre se excluye
Se permiten índices positivos y negativos.

Aplicación:

```
>>> palabra = 'ALGORITMOS'
>>> palabra [0:4]
ALGO

>>> palabra[0:-6]
ALGO

>>> palabra[-6:-1]
RITMO

>>> palabra[-8:-6]+palabra[-9,-8]
GOL

>>> palabra[-6:0]*2
RITMOSRITMOS

>>> palabra[-1+1]
```

Funciones básicas aplicadas a cadenas de texto

STR () convierte un dato numérico en string.

LEN() cantidad de caracteres que contiene la cadena

MIN() Devuelve el primer carácter en orden alfabético, del argumento string

MAX() Devuelve el último carácter en orden alfabético del argumento string

C. Lógicos:

verdadero = **True**; falso = **False**

- Operadores relacionales:

Símbolo	Significado	Ejemplo	Resultado
<code>==</code>	Igual que	<code>5 == 7</code>	<code>False</code>
<code>!=</code>	Distinto que	<code>"rojo" != "verde"</code>	<code>True</code>
<code><</code>	Menor que	<code>8 < 12</code>	<code>True</code>
<code>></code>	Mayor que	<code>12 > 7</code>	<code>True</code>
<code><=</code>	Menor o igual que	<code>12 <= 12</code>	<code>True</code>
<code>>=</code>	Mayor o igual que	<code>4 >= 5</code>	<code>False</code>

Operadores lógicos – Tablas de Verdad

Operadores booleanos

► Operador And

x	y	x and y
V	V	V
V	F	F
F	V	F
F	F	F

► Operador Not

x	Not x
V	F
F	V

► Operador Or

x	y	x or y
V	V	V
V	F	V
F	V	V
F	F	F

Ejemplos:

Operador	Ejemplo	Explicación	Resultado
and	5 == 7 and 7 < 12	False and True	False
and	9 < 12 and 12 > 7	True and True	True
and	9 < 12 and 12 > 15	True and False	False
or	12 == 12 or 15 < 7	True or False	True
or	7 > 5 or 9 < 12	True or True	True
xor	4 == 4 xor 9 > 3	True o True	False
xor	4 == 4 xor 9 < 3	True o False	True